Systems Programming in C

Complete study notes - Volume 2

Covers every slide in **SPC3 - Flow of Control** and **SPC4 - Functions**, with worked examples, diagrams, tables, and a practice set with answers.

KEY IDEA

How to read these notes. Content from your slides is the backbone. Green **BEYOND THE SLIDES** boxes add clarifying facts that are not on the slides, and are marked so you can tell the difference. Red **WATCH OUT** boxes flag common traps.

Contents

1. Flow of Control: Overview **5**

1.1 Blocks (compound statements) . . . 5

1.2 Comparison and logical operators . . . 6

2. Branching **6**

2.1 if and if-else . . . 6

2.2 The ternary operator ?: . . . 7

2.3 The dangling else . . . 8

2.4 Multiway branching with else if . . . 8

2.5 switch . . . 8

3. Loops and Jumps **10**

3.1 while . . . 10

3.2 do-while . . . 10

3.3 for . . . 11

3.4 The comma operator . . . 11

3.5 Declaring variables in for (C99) . . . 12

3.6 Example: sum of odd integers . . . 12

3.7 break . . . 13

3.8 continue . . . 13

3.9 The forbidden one: goto . . . 13

4. Self-Study Slides ("Study the remaining slides yourself") **14**

4.1 Operators so far: precedence and associativity . . . 14

4.2 Shortcut (short-circuit) evaluation of logical expressions . . . 15

4.3 Avoid these common pitfalls . . . 15

4.4 Comparing if-then in C and Python . . . 16

5. Functions: Building Blocks for Larger Programs **18**

5.1 The black-box view . . . 18

5.2 Function definitions . . . 18

5.3 Function declarations (prototypes) . . . 19

5.4 Example: computing b to the power n . . . 20

6. Static and Global Variables **21**

6.1 Be cautious with static and global variables . . . 21
6.2 Example: a static variable in a function . . . 21
6.3 Opinions about global and static variables . . . 23
7. Function Call Context and Memory Organization 23
7.1 Function call context . . . 23
7.2 Memory organization . . . 24
7.3 Zooming in on the stack . . . 25
8. Recursion 25
8.1 Stack frames during power(2, 8) . . . 26
9. Self-Study Slides ("Study the rest of the slides yourself") 27
9.1 Example: variable scope across files . . . 27
9.2 Order of evaluation of arguments . . . 28
9.3 Macros as "fast functions" . . . 28
9.4 Variable numbers of arguments . . . 29
10. Quick Reference Sheet 30
Control-flow constructs . . . 30
Functions and memory . . . 30
Gotchas checklist . . . 31
11. Practice Questions 32
Answers . . . 33

PART 1

Flow of Control

SPC3 (ABC Chapter 4): blocks, branching, loops, switch, break / continue, and common pitfalls

1. Flow of Control: Overview

Statements are **normally executed sequentially**, one after another. For **selective or repeated execution** C has "all the usual suspects" from Java / C++ / Python:

- **blocks**
- `if` and `if-else`
- `while`
- `for`
- `switch`
- `break`
- `continue`

The slide set adds `do-while`, the ternary operator `?:`, the comma operator, and a warning about `goto`.

1.1 Blocks (compound statements)

```
{
    int a, b;
    a = 3;
    b = a * 10;

    // C99
    int c;
    c = a * b;
}
```

- A block is a **list of statements enclosed by `{` and `}`**.
- It is **considered a single statement** - that's why it can be used wherever one statement is expected (the body of an `if`, a loop, ...).
- **No semicolon after the closing `}`.**
- It **can be empty**, and it **can be nested** (a block in a block).
- Useful for **branching and loop statements**.
- You can **define variables at the beginning of a block**. In **C99** you can also **mix declarations and code** (as `int c;` above).

> **BEYOND THE SLIDES**
> Variables declared inside a block are **local to it**: they exist only until the closing `}` (their *scope*). That is why blocks are handy for short-lived temporaries.

1.2 Comparison and logical operators

Family	Operators	Notes
Comparison	`== != > < >= <=`	Compare two expressions. **Pay attention to types!**
Logical	`&& || !`	AND, OR, NOT

- The result is **either 0 or 1, of `int` type**.
- Again: **0 means false, 1 means true** (and for *testing*, any non-zero value counts as true).

```
int a = (3 < 5);      // a is 1
int b = (3 == 5);     // b is 0
int c = !7;           // c is 0   (7 is "true", so ! gives false)
```

WATCH OUT
Chained comparisons don't work like in Python. `1 < 2 < 3` is `(1 < 2) < 3` = `1 < 3` = 1, but `3 > 2 > 1` is `(3 > 2) > 1` = `1 > 1` = **0**. Write `a < b && b < c` instead.

BEYOND THE SLIDES
"Pay attention to types": comparing a signed and an unsigned value converts the signed one to unsigned first, so `-1 < 1u` is **false**. Also avoid `==` on floating-point values; they are approximations.

2. Branching

2.1 `if` and `if-else`

```
if (<exp>)
    <stmt>
```

```
if (<exp>)
    <stmt1>
else
    <stmt2>
```

The slide shows these as flowcharts. In words:

- `if (exp) stmt`: evaluate `exp`. If it is **non-zero**, execute `stmt`. If it is **zero**, skip it and continue after the `if`.
- `if (exp) stmt1 else stmt2`: if `exp` is **non-zero** execute `stmt1`; if it is **zero** execute `stmt2`. Exactly one of the two runs.

KEY IDEA
`exp` is typically a comparison or logical expression, **but can be ANY expression** (float/double, pointer, ...). Zero means false; everything else means true.

- The statements can be **compound statements (blocks)**.
- ...**or other `if` statements!**
- **Beware of the dangling else**: an `else` matches the **nearest preceding `if`**. Use blocks to disambiguate (see section 2.3).

Example: min

```
int i, j, min;

if (i < j)
    min = i;
else
    min = j;

// Indentation is not required; the above 4 lines are the same as
if (i < j) min = i; else min = j;
```

Example: if-else with blocks

```
int i, j, k;

if (i < j) {
    k = i;
    printf("i is selected.\n");
}     // no ; here
else {
    k = j;
    printf("j is selected.\n");
}
```

Both branches contain **two statements**, so each needs braces. Note the comment on the slide: **no `;` after the `}`**.

2.2 The ternary operator `?:`

The only C operator with **three operands**:

```
exp1 ? exp2 : exp3
```

- `exp1` **is evaluated first.**
- If `exp1` is **non-zero (true)**, `exp2` is evaluated and **its value is the value of the whole ternary expression**.
- If `exp1` is **zero (false)**, `exp3` is evaluated and **its value is used instead**.

```
min = i < j ? i : j;       // same effect as the if-else "min" example
```

KEY IDEA

Unlike `if-else` (a *statement*), the ternary is an **expression**: it produces a value, so it can sit inside a larger expression or on the right of `=`.

BEYOND THE SLIDES

Only one of `exp2` / `exp3` is ever evaluated. In the precedence table it sits just above the assignment operators and associates **right to left**.

2.3 The dangling else

```
if (a) if (b) s1++; else s2++;      // Avoid this

// which 'if' is the 'else' associated with?

if (a) {if (b) s1++; else s2++;}    // option 1
if (a) {if (b) s1++;} else s2++;    // option 2

// C chooses option 1
```

C always pairs `else` with the nearest unmatched `if` (option 1: the `else` belongs to `if (b)`), no matter how you indent. To get option 2 you must add the braces yourself. Best practice: **always use braces** when nesting.

a	b	Option 1 (what C does)	Option 2 (what the indentation might suggest)
1	1	`s1++`	`s1++`
1	0	`s2++`	nothing
0	(any)	**nothing**	`s2++`

2.4 Multiway branching with `else if`

```
// Assume all variables are defined as int
if (i == 0)
    n0++;
else if (i == 1)
    n1++;
else if (i == 2)
    n2++;
else
    n_other++;
```

This is not a new construct, just an `else` whose statement is another `if`. The conditions are tried **top to bottom**; the first true one runs and the rest are skipped. The final `else` catches everything else.

2.5 `switch`

Also called a **"selection"** statement. It picks a starting point based on the value of an **integer expression**:

```
switch (<integer expression>) {
    case <integer constant1>:
        <statements>
    case <integer constant>:
    case <integer constant>:
        <statements>
    default:
        <statements>
}
```

- The controlling expression must have an **integer type** (`int`, `char`, ...).
- Each `case` label must be an **integer constant**.
- Control jumps to the **matching `case` label**, or to `default` if none matches.

- From there execution **keeps going, falling through** every following statement and label until a `break` or the closing `}`.

Switch example (from the slides)

```
// Assume all variables are defined as int
switch (i) {
    case 0:
        n0 ++; break;   // Note the break statement
    case 1:             // No break for case 1. Will continue.
    case 2:
    {   // Can put a block here and define new variables
        int a = d + 10;
        n1 = a * 10; break;
    }
    default:
        n_other ++;
}
```

Value of i	What runs
0	`n0++`, then `break` leaves the switch
1	no statements under `case 1:` - **falls through** into `case 2:`'s block: `a = d + 10; n1 = a * 10;` then `break`
2	the same block as for 1
anything else	`default`: `n_other++` (it's the last label, so nothing follows)

Two details from the slide comments: **a `case` body may be a block** so you can define new variables (e.g. `int a`), and `case 1:` deliberately has **no** `break`, so it shares the code of `case 2:`.

Switching: with and without `break`

The slides show this as flow diagrams. Without breaks, once you enter at a label you run **every block below it**; with a `break` at the end of each block you run **only that block** and then continue after the switch:

```
switch(expression) {                switch(expression) {
    case label0: <block0>               case label0: <block0>; break;
    case label1: <block1>               case label1: <block1>; break;
    ...                                 ...
    default:     <blockn>               default:     <blockn>
}                                   }
 no breaks: label1 runs block1,      with breaks: label1 runs block1 only,
 block2, ... blockn (falls through)  then control goes to the next block
```

KEY IDEA

The second form is **"usually what you mean"** (slide 28, with the banner **"Don't forget the breaks"**). Fall-through is legal, and sometimes intended (grouping `case 1:` and `case 2:`), but is a classic source of bugs.

BEYOND THE SLIDES

Duplicate `case` values are a compile error, and `default` is optional (it need not be last). You can't `switch` on a `double` or on a string; for those use an `if / else if` chain.

3. Loops and Jumps

3.1 `while`

```
while (<exp>)
    <stmt>
```

Test first, then run. If `exp` is zero, the body is skipped (possibly never executing); if non-zero, run the body and test again.

Example: computing the sum of 0..99.

```
int i = 0, sum = 0;
while (i < 100) {
    sum = sum + i;
    i++;
}

// Same as
while (i < 100)  sum += i++;
```

`sum += i++` uses the **old** value of `i` (post-increment), adds it to `sum`, then bumps `i` - so it is exactly the two-statement body. The result is 0 + 1 + ... + 99 = **4950**.

3.2 `do-while`

- **Checks the condition after executing the loop body.**
- So the body is **executed at least once**.

```
do
    <stmt>
while (exp);

// Example: computing the sum of 0..99
int i = 0, sum = 0;
do {
    sum = sum + i;
    i++;
} while (i < 100);
```

WATCH OUT
`do-while` **ends with a semicolon** after the `while (exp)`; `while` and `for` loops do not.

	while	do-while
Condition checked	**before** each iteration	**after** each iteration
Minimum iterations	0	1
Typical use	"keep going while ..."	"do it once, then maybe again" (e.g. re-prompt for valid input)

3.3 `for`

- Sometimes called the **"counting" loop**, but it is **more like a Swiss-army knife!**
- **Three expressions:** initialization, condition, increment.

```
for ( <exp1>; <exp2>; <exp3> )
    <stmt>

// is equivalent to
<exp1>;
while (<exp2>) {
    <stmt>
    <exp3>;
}
```

The slide's flowchart: **evaluate** `exp1` once; then repeatedly **test** `exp2` (zero: exit), run `stmt`, **evaluate** `exp3`, and test again.

> **KEY IDEA**
> Slide note: `for (;;)` **is an infinite loop** - an empty condition counts as always true (you leave it with `break` or `return`).

Computing the sum of 0..99 with `for` - four ways

```
int i, sum;

// one way
sum = 0;
for (i = 0;  i < 100;  i++) sum = sum + i;

// another way, with all initializations inside
for (sum = i = 0;  i < 100;  i++) sum += i;

// yet another way, with empty body
for (sum = i = 0;  i < 100;  sum += i++);

// yet another, using comma operator
for (sum = 0, i = 0;  i < 100;  sum += i, i++);
```

All four compute 4950. Things to notice: `sum = i = 0` is a **chained assignment** (right to left; see SPC2); a lone `;` **is an empty statement**, so the loop has an empty body and does its work in the increment slot; and the **comma operator** lets one slot hold two expressions.

> **WATCH OUT**
> A stray `;` right after `for (...)` or `while (...)` makes the body **empty** and is a common bug: `for (i = 0; i < n; i++);` followed by a block runs the block only once, after the loop.

3.4 The comma operator

```
exp1 , exp2
```

- Takes **two expressions**. `exp1` **is evaluated first, then** `exp2`**.**
- `exp2` **is the result** of the whole expression.

- It has the **lowest precedence** of all operators and **associates left to right**: `exp1, exp2, exp3` is `(exp1, exp2), exp3`.

Order can make a difference. These two loops are *not* the same:

```
for (sum = 0, i = 0;  i < 100;  sum += i, i++);     // adds 0,1,...,99   -> 4950
for (sum = 0, i = 0;  i < 100;  i++, sum += i);     // adds 1,2,...,100  -> 5050
```

In the second, `i` is incremented *before* it is added, so the last value added is 100 (the loop test still stops it from running again because `i` is then 100).

WATCH OUT
The commas in a **function call** `f(a, b)` or a **declaration** `int i, j;` are *separators*, not the comma operator.

3.5 Declaring variables in `for` (C99)

```
// C99 allows variable declarations in init expression
// i lives in the loop. It is undefined after the loop

int  sum = 0;
for (int i = 0; i < 100; i ++)
    sum += i++;
```

- **C99** lets you declare the loop variable **inside** the `for` header.
- `i` **lives only in the loop**; using it after the loop is an error (it is out of scope).

WATCH OUT
In this slide's example, `i` is incremented **twice** per iteration (`i++` in `sum += i++` and again in the header), so it adds only the even numbers 0, 2, ..., 98 and `sum` ends as **2450**, not 4950. The slide is demonstrating the scoping rule, not a sum.

BEYOND THE SLIDES
If your compiler complains, make sure it is in C99 mode or later (`gcc -std=c99`); current gcc defaults to a newer standard where it just works.

3.6 Example: sum of odd integers

```
int i, sum;

sum = 0;
for (i = 1; i < 100; i++)
    if (i % 2) sum += i;

// (i % 2) is the same as (i % 2 != 0)
// the condition can also be (i & 1)
```

`i % 2` is 1 (true) for odd numbers and 0 for even ones, so only odd `i` are added. The lowest bit of an odd number is 1, so `i & 1` is the bitwise way of saying the same thing (SPC2 masks). The sum 1 + 3 + ... + 99 is **2500**.

3.7 `break`

- **Most commonly used in `switch` statements**: it **prevents "fall-through"** to the next case.
- **Also works in loops** (`for`, `while`, `do-while`): the **loop execution is terminated immediately** and control resumes at the statement **immediately following the loop**.

```
{ // begin loop body
  ...
  break;       // jumps out past the end of the loop body
  ...
} // end loop body
...            // <- execution resumes here

switch (a) {
  ...
    break;     // jumps out past the end of the switch
  ...
} // end of switch
...
```

> **BEYOND THE SLIDES**
> In nested loops, `break` only exits the **innermost** enclosing loop (or switch).

3.8 `continue`

- **Skips the rest of the current loop iteration** and continues with the next one.
- Can be used within `for`, `while` and `do-while` loops, and can appear inside a nested `if / else`.
- In **nested loops** it applies to the **innermost** enclosing loop.
- In a `for` loop it goes to the evaluation of the **increment** expression (`exp3`).

```
{ // begin loop body
  ...
  continue;    // skip the rest of this iteration
  ...
} // end loop body
```

Statement	Effect on a loop	Works in
`break`	**Ends the whole loop** now; resume after it	loops and `switch`
`continue`	**Ends this iteration** only; go to the next (for `for`: run `exp3` first)	loops only

3.9 The forbidden one: `goto`

```
    goto <label>;
...
label:
    <block>
```

The slide's advice, in escalating order: **Forget you ever knew `goto` existed. This is a terrible control primitive. Don't ever use it. Really. Seriously. Nothing good will come out of it.**

BEYOND THE SLIDES

`goto` jumps to a label anywhere in the same function, which makes control flow impossible to follow ("spaghetti code"). Structured constructs - blocks, `if`, loops, `switch` - give you everything you need.

Caveat emptor ("buyer beware")

`continue` and `break` ... "are troublesome fellows" - **easy to misuse**. The slide's recommendation:

- **Use `break` with `switch`.**
- **Avoid `break` within loops.**
- **Avoid `continue` altogether.**

INSTRUCTOR NOTE

This is the instructor's style rule for the course, so follow it in your assignments even though `break`/`continue` in loops are legal and common in real-world C. Usually you can fold the exit condition into the loop's own test instead.

4. Self-Study Slides ("Study the remaining slides yourself")

4.1 Operators so far: precedence and associativity

This is the table from the slide, from **most** to **least** tight binding. It extends the SPC2 table with comparison, logical, ternary and comma:

Operators (highest precedence first)	Associativity
`()` `++` (postfix) `--` (postfix)	left to right
`+` (unary) `-` (unary) `++` (prefix) `--` (prefix)	right to left
`* / %`	left to right
`+ -`	left to right
`< <= > >=`	left to right
`== !=`	left to right
`&&`	left to right
`||`	left to right
`?:`	right to left
`= += -= *= /=` etc.	right to left
`,` (comma operator)	left to right

Consequences: comparison binds **looser than arithmetic** (`a + 1 < b` means `(a + 1) < b`), and `&&` binds **tighter than** `||` (`a || b && c` means `a || (b && c)`).

BEYOND THE SLIDES
This slide's table leaves out the bitwise and shift operators. Their place: `<< >>` sit between `+ -` and `< <= > >=`; then `&`, `^`, `|` come after `== !=` and before `&&`. That's why `x & mask == 0` is a bug - it parses as `x & (mask == 0)`. Write `(x & mask) == 0`.

4.2 Shortcut (short-circuit) evaluation of logical expressions

For `&&` and `||`, **evaluation stops as soon as the outcome (true or false) is known**:

```
exp1 && exp2
// Evaluate exp1. If false, exp2 is not evaluated.

exp1 || exp2
// Evaluate exp1. If true, exp2 is not evaluated.
```

This **makes it safe to write code like**

```
if (a != 0 && c == b/a) ...      // no divide by 0 error
```

If `a` is 0, the left side is false, so `b/a` is never evaluated. Order matters: swapping the operands would divide by zero.

Expression	Right side evaluated when...	Result
`exp1 && exp2`	`exp1` is **true**	1 if both are true, else 0
`exp1 || exp2`	`exp1` is **false**	1 if either is true, else 0

WATCH OUT
Don't put side effects in the right operand and rely on them happening: `if (x > 0 && y++ > 5)` does **not** increment `y` when `x <= 0`.

4.3 Avoid these common pitfalls

Pitfall	Example	What goes wrong
Confusing assignment with equality test	`x=8` vs. `x==8`	`if (x = 8)` assigns 8 and is always true
Off-by-1 errors in counting loops	`for (int i=0; i <= n; i++) ...`	`<=` runs **n+1** times; use `<` to run n times from 0
Confusing logical and bitwise ops	`x && y` vs. `x & y`	`&&` tests truth; `&` works on bits (`1 && 2` is 1, `1 & 2` is 0)
Forgetting `break` in a switch	`case 1: ... case 2: ...`	unwanted fall-through into the next case
Dangling else in nested if-else	`if (a) if (b) s1++; else s2++;`	`else` binds to the inner `if`

4.4 Comparing `if-then` in C and Python

- **Indentation carries no information in C.**
- **Block structure is explicit, using `{ }`.**
- The **condition must be in parentheses.**

```
// C version (one line, legal but hard to read)
if (n==0) return 1; else return n*fact(n-1);

// Easier to read
if (n == 0)
    return 1;
else
    return n * fact(n - 1);

# Python version
if n==0:
    return 1
else:
    return n * fact(n-1)
```

The first C line and the indented C version are **identical to the compiler**; only humans see a difference. In Python the indentation *is* the block structure.

PART 2

Functions

SPC4 (ABC Chapter 5): definitions and prototypes, storage classes, the call stack, memory layout, recursion, scope, and macros

5. Functions: Building Blocks for Larger Programs

- **Effective programming requires problem decomposition** into manageable pieces.
- **A C program is made of functions**, starting from `main()`.

5.1 The black-box view

The idealized view: **a function is a black box**.

- It **SPECIFIES what** the computation will do.
- It **ABSTRACTS AWAY how** the computational process works.
- It takes **INPUTS** (e.g. Input 1, Input 2, Input 3) and produces an **OUTPUT**.

> **KEY IDEA**
> This continues the "procedural" idea from SPC1: the caller only needs to know **what** a function does and what it needs, never **how** it does it. That separation is what lets you build large programs from small, testable pieces.

5.2 Function definitions

```
type function_name ( parameter_list ) {
    declarations & statements
}
```

- **No nesting**: you cannot define a function inside another function.
- The **return type can be** `void` (no return value expected).
- If the return type is **missing, the compiler assumes** `int`.
- Execution **also terminates if the end of the function body is reached**; then the **returned value is undefined**.

Return statements

```
return;        // terminate execution and return control to caller
return expr;   // terminate and pass value of expr back to caller
```

Form	Used in	Meaning
`return;`	`void` functions	Leave the function now; there is no value to hand back
`return expr;`	functions with a return type	Leave the function; `expr` **(converted to the return type)** becomes the value of the call

> **WATCH OUT**
> Falling off the end of a **non-void** function and then *using* its result is a bug: the value is **undefined**. Make sure every path returns a value.

BEYOND THE SLIDES
The "missing type means `int`" rule is old C (K&R / C89). **C99 removed implicit `int`**, and compilers warn about it, so always write the return type. `main` is a special case: in C99, reaching its closing `}` acts like `return 0;`.

5.3 Function declarations (prototypes)

- Functions can be **defined in any order**.
- **Declare a function before its first use** if its definition comes later.
- Prototypes are often placed in **header files** (and reused).

```
#include <stdio.h>

int fahrToCelsius(int);                  // Declaration (prototype)

int main() {
   for(int fahr=0; fahr <= 300; fahr += 10)
      printf("%d F is %d C degrees\n", fahr, fahrToCelsius(fahr));
   return 0;
}

int fahrToCelsius(int degF) {            // Definition
   return 5 * (degF - 32) / 9;
}
```

	Declaration (prototype)	Definition
What it gives the compiler	The function's **name, return type and parameter types**	All of that **plus the body** (the actual code)
How many times	Any number of times (must agree)	**Exactly once** in the program
Ends with	a `;`	a block `{ ... }`

Here `main` calls `fahrToCelsius` *before* its definition appears, so the prototype tells the compiler what the call should look like. The parameter name is optional in a prototype (`int fahrToCelsius(int);`).

BEYOND THE SLIDES
Output starts `0 F is -17 C degrees` (5 * -32 / 9 = -160 / 9 = -17, since integer division truncates toward zero) and ends `300 F is 148 C degrees`. Also, `int main()` with empty parentheses means "unspecified arguments" in C; writing `int main(void)` (as in SPC1) is the precise form.

5.4 Example: computing b to the power n

```
int power(int b, int n)
{
   int rv = 1;
   while (n>0) {
      rv *= b;
      n--;
   }
   return rv;
}

int foo ()
{
   int i = 10;
   return power(2, i);
}
```

The power function:

- **Returns an `int`.**
- **Two parameters:** base `b` and exponent `n`, both `int`.
- **Functions can declare local variables** (here `rv`).
- **Parameters and local variables can only be accessed inside the function.**
- Their storage class is `auto` **by default**: they are **discarded when the function returns**.
- **Parameters are passed by value.** `n` **is changed** inside `power` (it counts down), but `i` **in** `foo` **DOES NOT change**.

Call-by-value means the function receives a **copy** of each argument. Tracing `power(2, 10)` called from `foo`:

Iteration	rv (before)	n (before)	rv after rv *= b	n after n--
1	1	10	2	9
2	2	9	4	8
3	4	8	8	7
...	...	...	...	...
10	512	1	1024	0

The loop ends when `n` is 0 and returns **1024**. Meanwhile `foo`'s `i` is still 10, because `power` only ever touched its own copy.

> **WATCH OUT**
> Since the function gets a copy, **a function cannot change the caller's variable by assigning to its parameter.** (To modify the caller's variable you pass its *address* - pointers, later in the course.)

6. Static and Global Variables

Beyond ordinary ("auto") locals there are three other kinds of variable:

Kind	Where declared	Lifetime	Visibility
Local (auto)	inside a function	only during the call; **discarded on return**	only inside that function
Static local	inside a function, with `static`	**entire program run** (retains its value across calls)	**not visible outside the function**
Global	outside all functions	**entire program run**	all functions; storage class `extern` **by default**, so **other files can access it**
Static global	outside all functions, with `static`	entire program run	**only functions in the same file, following the variable's declaration**

- **Static local variables:** not visible outside the function, but **retain their value across function calls**.
- **Global variables:** declared outside functions; **retained for the entire duration of the program**; storage class is `extern` **by default**, so they can be accessed from functions in other files.
- **Static global variables: visible only in functions defined in the same file following the variable declaration.**

> **KEY IDEA**
> The keyword `static` means two different-looking things: on a **local** it means "keep my value between calls"; on a **global** it means "keep me private to this file." In both cases the variable lives for the whole program.

6.1 Be cautious with static and global variables

- **"Nice" functions only depend on their inputs.**
- Static and global variables have **side-effects**: they **retain values across function calls** and so **change the meaning of the function at each call!**
- **You cannot understand the function without holding ALL the code in your head.**

> **WATCH OUT**
> The slide's rule (with a big WARNING sign): **unless you have really good reasons and really know what you are doing, do not use static or global variables.** The slide notes add: "Any deviation from the gospel is dangerous."

6.2 Example: a static variable in a function

```
int silly(int x) {
   static int hidden = 0;
   return x * 2 + hidden++;
}
```

- The "silly" function contains a **static declaration**.
- `hidden` **is set to 0 only the first time** `silly` **is executed.**
- **Its value is retained from call to call.**
- **The function output no longer depends on its inputs alone!**

Call	IN (x)	OUT	hidden before	hidden after
1st	1	2 (1*2 + 0)	0	1
2nd	1	3 (1*2 + 1)	1	2
3rd	1	4 (1*2 + 2)	2	3
4th	2	7 (2*2 + 3)	3	4

The same input (1) produced three different outputs. Because `hidden++` is a post-increment, the old value is used in the result and then it grows.

"It could be worse"

```
static int hidden = 0;

int silly(int x) {
   return x * 2 + hidden++;
}

void innocuous(int z) {
   hidden = hidden * z + 10;
}
```

- Now `hidden` **is declared outside** `silly` (a static global).
- Its value is still retained call to call, and the output still no longer depends on inputs alone.
- **Worse:** `hidden` **can be changed by other functions!** An innocent-looking `innocuous(z)` silently alters what `silly` returns later.

Call	IN (x)	OUT	hidden before	hidden after
1st	1	2	0	1
2nd	1	3	1	2
3rd	1	4	2	3
4th	2	7	3	4

(That is the table on the slide, which shows `silly` calls only.) If `innocuous(2)` were called between the 3rd and 4th calls, `hidden` would become 3 * 2 + 10 = 16 and the 4th call would return 2 * 2 + 16 = **20** instead of 7.

6.3 Opinions about global and static variables

The slides close with a counterpoint: *"Global Variables Are OK, and In Fact Necessary"* (an article by Norm Matloff, linked from the slide). The course will **deal with globals a lot later**. A noted fact: **actually, the stack and heap are global** (in the sense of being shared program-wide memory).

Legitimate examples from the slide:

- **Counting the number of times a function is called.**
- **Coefficients in an FFT** (a fixed table shared by calls).

> **INSTRUCTOR NOTE**
> Reconcile with the previous warning: default to **no** statics/globals; use them deliberately for genuinely program-wide state, and keep them as private (`static` at file scope) as you can.

7. Function Call Context and Memory Organization

7.1 Function call context

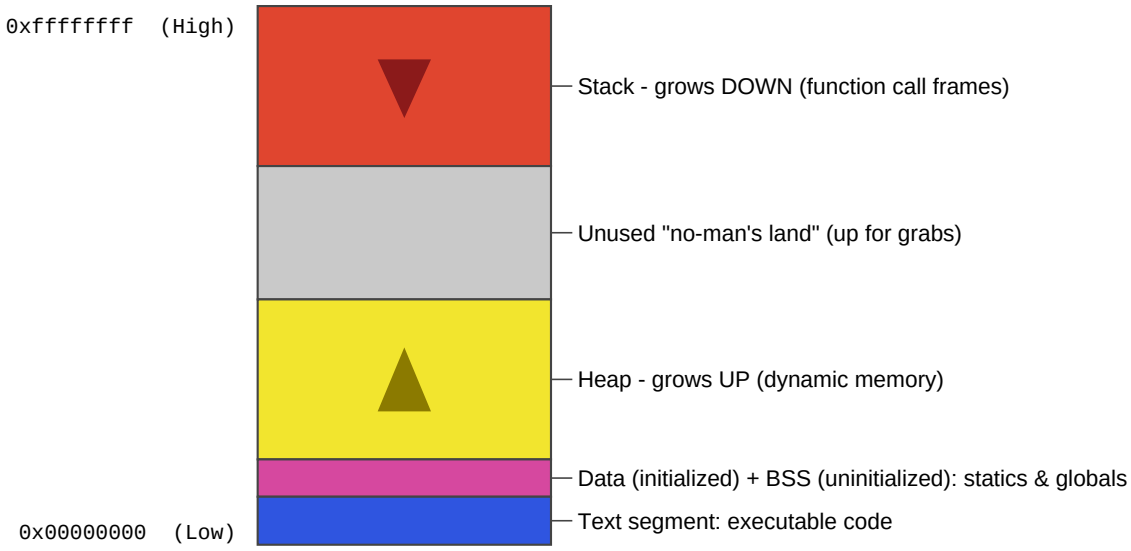
When a function runs, its **call context** includes:

- **Copies of the function arguments** (call-by-value).
- **(Auto) local variables.**
- **The return address**, ... (where to continue in the caller).
- Call contexts are **managed automatically using the execution stack**.
- A **stack frame is created automatically for each function call**.
- The frame **lasts for the duration of the call** and is **discarded automatically when the function terminates**.
- **NOTHING in the frame survives the call.**

> **KEY IDEA**
> "Nothing survives the call" is why a local variable's value is gone when the function returns, and why returning the address of a local variable is a serious bug (pointers, later).

7.2 Memory organization

Every process has an address space. The slides build this picture one layer at a time (low addresses at the bottom, high at the top):



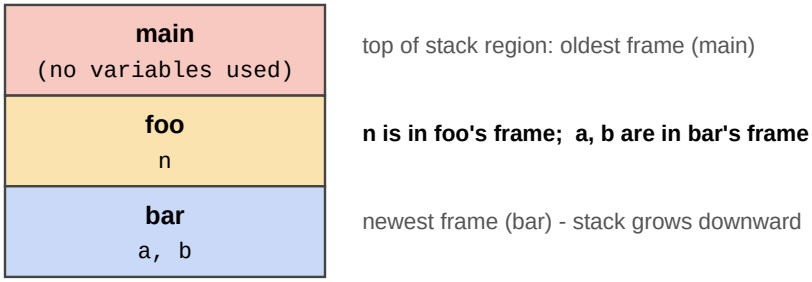
Region	Position	Contents
Text segment	bottom ("not exactly from address 0")	**Executable code**
Data segment	just above text	**Initialized** statics and globals
BSS segment	just above data	**Uninitialized** statics and globals
Heap	above statics; **grows from the bottom going up**	Dynamically allocated memory (later in the course)
Stack	**at the top, going down!**	Stack frames: arguments, auto locals, return addresses
Gray area	between heap and stack	**"No-man's land"** - up for grabs

Two caveats from the slide: the **low end may not start from 0**, and the **high end may not be** `0xff...ff` - this is a simplified model.

> **BEYOND THE SLIDES**
> Because the heap grows up and the stack grows down into the same gap, running out of room in the middle means **stack overflow** (e.g. runaway recursion) or heap exhaustion. Where each of your variables lives: local -> stack, `static` local or global -> data/BSS, `malloc` result -> heap, your code -> text.

7.3 Zooming in on the stack

```
int main()           { return foo(10); }
int foo(int n)       { return bar(n-1, n*n); }
int bar(int a, int b){ return a + b; }
```



`main` calls `foo(10)`, `foo` calls `bar(9, 100)`, and `bar` returns 109, which `foo` returns, which `main` returns. `n` **is in** `foo`**'s frame;** `a` **and** `b` **are in** `bar`**'s frame.** Each new call pushes a frame; each return pops it, and the stack grows and shrinks like a pile of plates.

8. Recursion

Recursive calls are supported: a function may call itself. Here is `power` written two ways:

```
// iterative
int power(int base, int n) {
   int rv = 1;
   while (n>0) {
      rv *= base;
      n--;
   }
   return rv;
}

// recursive version
int power(int base, int n) {
   if (n==0)
      return 1;
   else {
      int p = power(base, n-1);
      return base * p;
   }
}
```

The recursive version is **induction on n**:

- **Base case:** `n == 0` returns **1** (anything to the power 0).
- **Inductive case:** the answer is `base` multiplied by `power(base, n-1)`.

WATCH OUT

Every recursion needs a **base case that is reached** and a call that **moves toward it** (`n-1`). Without them the function recurses forever and the stack overflows. (The two definitions above share a name only for comparison; a real program can have just one `power`.)

8.1 Stack frames during `power(2, 8)`

The slides animate this program across ten slides. The program:

```
#include <stdio.h>

int power(int base, int n) {
   if (n==0)
      return 1;
   else {
      int p = power(base, n-1);
      return base * p;
   }
}

int main(int argc, char* argv[])
{
   printf("%d^%d = %d\n", 2, 8, power(2,8));
   return 0;
}
```

Going down (calls). `main` holds `argc=1, argv=...`. It calls `power(2,8)`, which calls `power(2,7)`, and so on until `power(2,0)`. Each frame has its own `base`, `n`, and `p`, and `p` **is still unknown (`?`)** because each frame is waiting for the call beneath it.

```
main            argc=1, argv=...
power(2,8)      base=2, n=8, p=?
power(2,7)      base=2, n=7, p=?
power(2,6)      base=2, n=6, p=?
power(2,5)      base=2, n=5, p=?
power(2,4)      base=2, n=4, p=?
power(2,3)      base=2, n=3, p=?
power(2,2)      base=2, n=2, p=?
power(2,1)      base=2, n=1, p=?
power(2,0)      base=2, n=0, p=?        <- base case: returns 1, no more calls
```

Coming back (returns). The base case returns 1. Each frame then fills in `p` from the call below, returns `base * p`, and is **discarded**:

Frame (as it finishes)	p (value from the call below)	Returns base * p
`power(2,0)`	- (base case)	**1**
`power(2,1)`	1	2 * 1 = **2**
`power(2,2)`	2	2 * 2 = **4**
`power(2,3)`	4	2 * 4 = **8**
`power(2,4)`	8	2 * 8 = **16**
`power(2,5)`	16	2 * 16 = **32**
`power(2,6)`	32	2 * 32 = **64**
`power(2,7)`	64	2 * 64 = **128**
`power(2,8)`	128	2 * 128 = **256**

The slides' final frames show exactly this: `n=1, p=1`; `n=2, p=2`; `n=3, p=4`; `n=4, p=8`; `n=5, p=16`; `n=6, p=32`; `n=7, p=64`; `n=8, p=128`; then **returned value = 256** and the output:

```
2^8 = 256
```

> **KEY IDEA**
> At the deepest point there are **10 frames** on the stack: `main` plus nine calls to `power` (n = 8 down to 0). The recursion depth is therefore n + 1 calls. Each frame has its **own independent copy** of `base`, `n` and `p`, which is why the same code can be "in progress" nine times at once.

9. Self-Study Slides ("Study the rest of the slides yourself")

9.1 Example: variable scope across files

```c
/* a.c */
static int i;
int j;

int foo(int n)
{
  int rv;
  static int k;
  // can access i, j, k
}

int bar(int n)
{
  // can access i, j
}
```

```c
/* b.c */
// declare j is external
// note the keyword extern
extern int j;
// Cannot define another j

// Can define another i
// this is a different i
static int i;

int f1(void)
{
  // can access j (in a.c)
  j++;
}
```

Name	Defined in	Kind	Who can access it
`i` (a.c)	a.c	static global	functions in a.c **after** its declaration (`foo`, `bar`)
`j`	a.c	global (`extern` by default)	`foo`, `bar` in a.c; **b.c too, via** `extern int j;`
`k`	inside `foo`	static local	**only** `foo` (but keeps its value between calls)
`rv`, `n`	inside `foo`	auto local / parameter	only `foo`, during one call
`i` (b.c)	b.c	static global	functions in b.c; **a different variable from a.c's** `i`

- `extern int j;` in `b.c` is a **declaration**, not a definition: it says "a `j` exists elsewhere". b.c **cannot define another** `j` or the linker would find two.
- Each file **may define its own** `static int i`: because they are `static`, they are different, private variables with no clash.

BEYOND THE SLIDES
`extern` is to variables what a prototype is to functions: it declares without defining. Put such declarations in a shared header.

9.2 Order of evaluation of arguments

```
i = 1;
foo(i++, i++);      // calling foo
```

Is it `foo(1, 2)` or `foo(2, 1)`? **Do not assume one way or another.**

WATCH OUT
C does **not specify the order in which a function's arguments are evaluated**. Two `i++` in one argument list makes the result unpredictable (and technically undefined behavior). Never use a variable twice in the same call if one use modifies it; do it in separate statements instead.

9.3 Macros as "fast functions"

Macros can take arguments! They are typical for implementing MIN / MAX:

```
#define MIN(x,y) ((x) <  (y) ? (x) : (y))
#define MAX(x,y) ((x) >= (y) ? (x) : (y))

int main()
{
    int a = 10, b = 20;
    int x = MIN(a,b);
}
```

Notice how arguments are always put in parentheses. Why? The next slide answers:

```
#include <stdio.h>
#define MULG(x,y) ((x)*(y))
#define MULB(x,y) (x*y)

int main()
{
    int x = MULG(99+1,2);
    int y = MULB(99+1,2);
    printf("x is %d\n",x);
    printf("y is %d\n",y);
    return 0;
}

$ cc macros.c ; ./a.out
x is 200
y is 101
```

Macro	Call	Text after preprocessing	Value
`MULG(x,y) ((x)*(y))` (G = good)	`MULG(99+1,2)`	`((99+1)*(2))`	**200**
`MULB(x,y) (x*y)` (B = bad)	`MULB(99+1,2)`	`(99+1*2)`	**101** (`*` binds tighter than `+`)

KEY IDEA

A macro is **text substitution** done by the preprocessor *before* compilation - it doesn't evaluate arguments first. So wrap **each parameter** in parentheses **and the whole body** too, or operator precedence will rearrange your expression.

BEYOND THE SLIDES

Macros are "fast" because there is no function call, but they have no type checking and may evaluate an argument **more than once**: `MIN(i++, j)` can increment `i` twice. Remember the macro tools `#define` / `#include` from the preprocessing stage in SPC1's compilation figure.

9.4 Variable numbers of arguments

`printf` can be called with different numbers of arguments:

```
printf("Hello");
printf("Hello, %s!", name);
```

How can you pass different numbers of arguments? With an ellipsis `...` in the parameter list:

```
int foo(int n, ... );     // prototype
int foo(int n, ... )      // implementation
{
    // use functions in <stdarg.h> to access the arguments
    // va_start(), va_arg()
}
```

- The `...` must come **last**, after at least one named parameter (here `n`, often used to tell the function how many extras to expect).
- The `<stdarg.h>` header provides the access macros `va_start()` and `va_arg()`.

BEYOND THE SLIDES

The full set is `va_list`, `va_start`, `va_arg`, and `va_end` (to clean up). On the slide the `printf` strings use curly "smart quotes" - real code needs straight `"` quotes.

10. Quick Reference Sheet

Control-flow constructs

Construct	Syntax	Key point
if	`if (e) s`	runs `s` when `e` is non-zero
if-else	`if (e) s1 else s2`	exactly one runs; else pairs with nearest if
ternary	`e1 ? e2 : e3`	an expression with a value
while	`while (e) s`	test first; may run 0 times
do-while	`do s while (e);`	body runs at least once; note the final `;`
for	`for (a; b; c) s`	`a; while (b) { s c; }`; `for(;;)` is infinite
switch	`switch (int) { case K: ... }`	falls through unless `break`
break	`break;`	leaves innermost loop / switch
continue	`continue;`	next iteration of innermost loop (for: runs `c` first)
comma	`e1, e2`	lowest precedence; value of `e2`

Functions and memory

Item	Rule
Definition	`type name(params) { ... }`; **no nested functions**
Prototype	`type name(types);` before first use; usually in a header
Arguments	**Passed by value** (callee gets copies)
Local (auto)	In the stack frame; **gone after return**
`static` local	Keeps its value across calls; private to the function
Global	Lives the whole run; `extern` **by default** (other files can see it)
`static` global	File-private
Stack / heap	Stack at top growing down; heap grows up from below the gap
Data / BSS / Text	Initialized statics / uninitialized statics / code
Macros	Text substitution; parenthesize every parameter and the whole body

Gotchas checklist

- `=` assigns, `==` compares; `&&` / `||` are logical, `&` / `|` are bitwise.
- `1 < 2 < 3` is not a range test; use `&&`.
- Dangling else: it binds to the nearest `if`. Use braces.
- No `;` after a block `}`, but **do** put `;` after `do { } while (e)`.
- A stray `;` after `for (...)` / `while (...)` gives an empty body.
- Off-by-one: `i <= n` vs. `i < n`.
- Missing `break` in `switch` causes fall-through.
- Order of evaluation of function arguments is unspecified.
- A function can't change the caller's variable through a parameter (pass by value).
- Don't use values from a non-void function that fell off the end without `return`.

11. Practice Questions

1. What are the values of `1 < 2 < 3` and `3 > 2 > 1`?

2. `if (a) if (b) s1++; else s2++;` - with `a = 0, b = 1`, which counter changes?

3. What does `min = i < j ? i : j;` do when `i = 4, j = 9`?

4. How many times does the body run for `while (x) {...}` and `do {...} while (x);` if `x` is 0 at the start?

5. Rewrite `for (a; b; c) s` using `while`.

6. What are the final values of `sum` for `for (sum=0,i=0;i<100;sum+=i,i++);` and `for (sum=0,i=0;i<100;i++,sum+=i);`?

7. In the slide's `switch (i)` example, what runs for `i = 0`, `i = 1`, `i = 5`?

8. Why is `if (a != 0 && c == b/a)` safe from division by zero?

9. What does `continue` do inside a `for` loop? What does `break` do?

10. What is the course's style rule about `goto`, `break` and `continue`?

11. Why does `power(2, i)` not change `i` even though `power` decrements `n`?

12. Where does each live: a local variable, a `static` local, an initialized global, an uninitialized global, a `malloc`ed block, the function's code?

13. How many stack frames exist at the deepest point of `power(2,8)` called from `main`, and what does `power(2,3)` return?

14. Call `silly(1), silly(1), silly(1), silly(2)` with `static int hidden = 0` and `return x*2 + hidden++`. What are the outputs?

15. What do `MULG(99+1,2)` and `MULB(99+1,2)` evaluate to, and why do they differ?

16. What is the difference between `static int i;` at file scope and `int j;` at file scope, as seen from another file?

17. Is `foo(i++, i++)` equivalent to `foo(1,2)`?

Answers

1. `1` and `0`. Comparisons associate left to right: `(1<2)<3` is `1<3` = 1; `(3>2)>1` is `1>1` = 0.

2. Neither. The `else` pairs with `if (b)`, and the inner `if` is never reached because `a` is 0.

3. It evaluates `i < j` (true), so the value is `i` and `min = 4`.

4. `while`: 0 times. `do-while`: exactly 1 time.

5. `a; while (b) { s c; }` (note: `continue` in the `for` jumps to `c`).

6. 4950 and 5050. The second adds `i` after incrementing it, so it adds 1..100.

7. `i=0`: `n0++`. `i=1`: falls through into the `case 2` block, so `a = d + 10; n1 = a * 10;`. `i=5`: `default`, `n_other++`.

8. `&&` short-circuits: if `a != 0` is false, `c == b/a` is not evaluated.

9. `continue` skips the rest of this iteration and runs the increment expression, then tests the condition. `break` leaves the loop entirely.

10. Never use `goto`; use `break` with `switch` but avoid it in loops; avoid `continue` altogether.

11. Arguments are passed by value, so `n` is a copy of `i`; only the copy changes.

12. Stack; data (if initialized) or BSS (if not); data; BSS; heap; text segment.

13. 10 frames (`main` + 9 power calls, n = 8..0). `power(2,3)` returns 8.

14. 2, 3, 4, 7.

15. 200 and 101. `MULB` expands to `(99+1*2)`; without parentheses around `x`, `*` binds tighter than `+`.

16. `j` is `extern` by default: another file can use it with `extern int j;`. `static int i` is private to its own file.

17. You cannot assume that: the order in which arguments are evaluated is unspecified, so it might be `foo(2,1)` (and it is undefined behavior). Don't write it.